

CROWD SAFETY & PILGRIM MANAGEMENT PLATFORM

WariVerse

Solution Document & Working of the System

A million people walk 250 km over eighteen days to reach a temple built for a few thousand. WariVerse is the system that sees the crowd forming, says what to do about it before it forms, and keeps working when its own parts fail.

Deployment context Shri Vitthal-Rukmini Temple, Pandharpur, Maharashtra

Event Ashadhi Ekadashi Wari — the annual Palkhi procession

Scope 8 functional modules · 10 build phases · 106 API endpoints

Compliance frame Digital Personal Data Protection Act, 2023 (India)

1 Executive summary

What the problem is, what WariVerse does about it, and the one design idea the whole system turns on.

The Wari is one of the largest recurring pedestrian gatherings on earth. Roughly a million pilgrims converge on Pandharpur — a town of about a hundred thousand — after an eighteen-day walk from Alandi and Dehu. The temple's darshan corridor can move a few thousand people an hour. The arithmetic does not work, and the gap between arrival rate and service rate is where crowd crushes happen.

WariVerse manages that gap from both sides at once. It reduces the arrival rate with time-slotted darshan passes that reslot themselves when the gate runs behind, and it watches the crowd continuously so operators act on a *developing* situation rather than a developed one. Around those two it adds the operational machinery a temple trust actually needs: incident dispatch with SLA clocks, a tamper-evident ledger for queue breaches, an offline-first Marathi pilgrim app, a forecasting model, and tracking for the eighteen-day procession itself.

THE TECHNICAL INSIGHT THE SYSTEM IS BUILT ON

Crush events correlate with **stalled and opposing crowd flow**, not with raw density alone. A dense crowd that is moving is a queue; a dense crowd that has *stopped* moving is a crush precursor. WariVerse measures a **stagnation index** and a **counter-flow ratio** alongside density, and its most severe rule fires on the combination — which appears minutes before density alone crosses a critical band. Density-only systems alert late. That is the difference between a warning and a post-mortem.

What is delivered

Module	What it solves
M1 Smart Darshan Pass	Spreads arrivals across time slots; reslots downstream passes automatically when the gate runs behind
M2 Crowd Intelligence	Per-zone density, flow, stagnation, counter-flow from CCTV; alerts carrying a named recommended action
M3 Command Center	Live map, alert feed, KPI strip and replay scrubber for the control room
M4 Incidents & SOS	Pilgrim SOS, responder dispatch ranked by distance, SLA clocks, missing-person cases
M5 Queue Breach Ledger	Hash-chained, tamper-evident record of unauthorised gate entries
M6 Predictive Forecasting	30/60/90-minute density prediction with confidence intervals and rule-derived actions
M7 Pilgrim PWA	Offline-first Marathi app: pass, map, facilities, emergency numbers
M8 Palkhi & Dindi Tracking	Extends coverage from the one-day peak to the full eighteen-day walk
AI Pilgrim Assistant	Answers in Marathi/Hindi/English, strictly from live system data

2 The problem being solved

2.1 The physical situation

Pilgrims — *warkaris* — walk in organised groups called **dindis**, escorting the **palkhi** (palanquin) of a saint. The final approach and the darshan queue concentrate an eighteen-day flow into a few hundred metres of corridor over roughly one day.

Pressure	Consequence
~1,000,000 arrivals into a town of ~100,000	Every facility oversubscribed by an order of magnitude
Darshan throughput ~6,000/hour	Arrival rate exceeds service rate for hours at a stretch
Queues form in narrow corridors	Density concentrates exactly where escape routes are fewest
Mobile networks saturate	Any system needing 4G fails precisely when it is needed
Unauthorised entry at restricted gates	Queue integrity collapses; disputes become unresolvable
Families become separated	Missing persons is the highest-frequency real incident

2.2 Why the usual approaches fall short

- **Manual crowd control alone** reacts to what marshals can see — in a corridor, a few metres.
- **Density-only analytics** alerts after the dangerous state has formed, because density lags the stalled-flow condition that precedes a crush.
- **Ticketing without feedback** issues slots against a *planned* throughput and never corrects when the gate runs slow, so the queue absorbs the error.
- **Biometric systems** are undeployable by a temple trust under the DPDP Act — and unnecessary for a problem that is about counting, not identifying.

3 The solution: five design commitments

These five decisions shape every module. They are why the system behaves as it does under stress, and what separates it from a dashboard.

ONE Act on the precursor, not the event

Stagnation and counter-flow are first-class measured signals. The most severe rule (**R-M2-01**) fires on *high density AND high stagnation together* — a dense crowd that has stopped moving — because that combination appears before density alone crosses the critical band.

TWO Never present an estimate as a certainty

Every number carries a timestamp, a source (`live` / `video` / `sim`) and a confidence. A zone that lost its cameras reads as an estimate. A stale reading reads as **UNKNOWN**, never as clear — a zone that stopped being measured and an empty zone produce identical numbers, and confusing the two is how someone walks into a crush.

THREE Every recommendation comes from a numbered rule

Suggested actions come from a rule table, not a model. An operator about to hold a gate on ten thousand people sees *which rule* said so — the `rule_id` travels on every alert row. A language model may *rephrase* a recommendation; it may never *produce* one.

FOUR Degrade, never fail closed

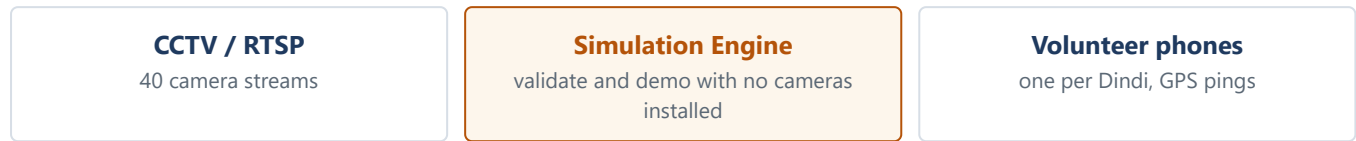
Every module has a documented manual mode. The AI service dying does not stop passes, scanning, SOS, incidents or the breach ledger. Redis dying costs freshness, not correctness. The assistant losing its model falls back to deterministic answers over the same data.

FIVE Zero-biometric by construction

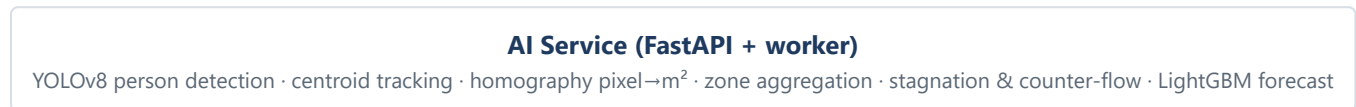
No facial recognition, no gait analysis, no biometric templates, no cross-camera re-identification — not as a configurable control but as an *absence*. There is no table that could hold a biometric template. This is what makes the system deployable by a temple trust under the DPDP Act.

4 System architecture

INPUTS

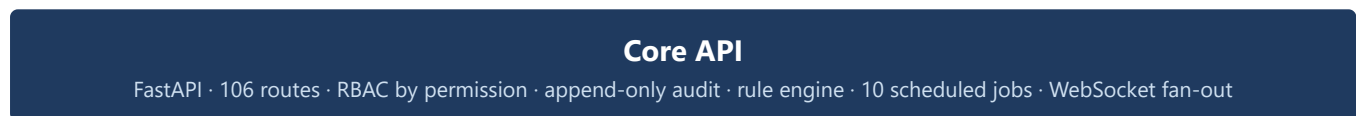


PROCESSING — MEASURES, NEVER STORES

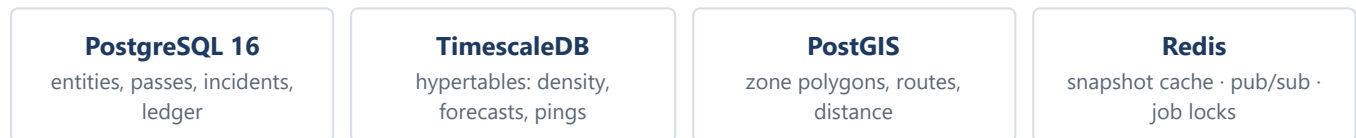


▼ events over HTTP + shared secret ▼

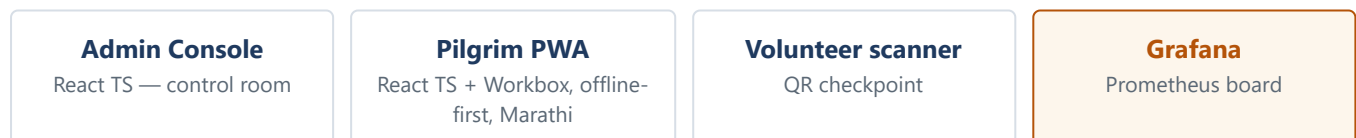
SYSTEM OF RECORD — THE ONLY WRITER



STORAGE



CLIENTS



THE BOUNDARY THAT MATTERS MOST

The AI service measures; the Core API is the only thing that records. The AI service holds no state and owns no table. It can be killed and restarted mid-Wari without losing a single reading, because it never held one. That single boundary is what makes the whole "degrade, never fail closed" property achievable rather than aspirational.

4.1 Technology stack

Layer	Technology	Why this one
Backend	Python 3.12, FastAPI, SQLAlchemy 2.0 (async)	Typed schemas at the boundary; async suits an I/O-bound event system
Database	PostgreSQL 16 + TimescaleDB + PostGIS	One database for relational, time-series and spatial — no second system to keep consistent
Cache / bus	Redis 7	Snapshot cache with a safety TTL, pub/sub fan-out, job locks
Vision	YOLOv8, OpenCV, homography	Person detection only; no recognition. Homography converts pixels to real ground area
Forecasting	LightGBM with prediction intervals	Interpretable, fast to retrain, and carries its own uncertainty
Frontend	React + TypeScript, Workbox	Offline-first service worker is a hard requirement, not a nicety
Assistant	Gemini via function calling (httpx)	Fixed tool list; thin client, no vendor SDK dependency tree
Observability	Prometheus, Grafana, JSON logs with trace_id	Per-endpoint <i>and</i> per-zone-pipeline metrics
Delivery	Docker Compose, Alembic, GitHub Actions	Whole stack from one command; bandit + pip-audit on every PR

5 How each module works

M1 Smart Darshan Pass

Solves: the arrival rate. Instead of everyone arriving at dawn, arrivals are spread across 30-minute slots sized from real throughput.

- **Slot capacity** is derived from configured throughput, not guessed: `capacity = throughput_per_hour × slot_minutes / 60`.
- **25% walk-in reserve** is subtracted *before* online booking opens. This protects pilgrims without smartphones from being priced out of darshan by digitally literate ones — equity enforced by arithmetic, not by policy.
- **QR passes are signed** with a per-day derived key and rotate on a short window, so a screenshot shared on WhatsApp does not scan twice.
- **Dynamic reslotting** — a job every 5 minutes compares planned throughput against measured scans. If the gate is running >20% behind, downstream passes shift and the holder's phone is notified with a revised slot.
- **Honest waits.** The pass screen shows "your slot 15:30, current wait 4h 20m" — computed from the queue actually ahead, and labelled an estimate.

M2 AI Crowd Intelligence

Solves: seeing the crowd. Turns camera frames into four numbers per zone every ten seconds.

- 1 **Detect.** YOLOv8 finds people in a frame. Persons only — no faces, no attributes.
- 2 **Track briefly.** Centroids are matched across a few frames to derive movement. Track IDs are *ephemeral and in-memory*, discarded after aggregation — this is what prevents re-identification by design.
- 3 **Project to the ground.** A four-point homography maps image pixels to real square metres. Without this calibration a density figure is fiction, so an uncalibrated zone is refused rather than estimated.
- 4 **Aggregate per zone.** Person count, density, flow vector, **stagnation index** (how much of the crowd has stopped) and **counter-flow ratio** (opposing streams colliding).
- 5 **Publish.** The reading is posted to Core API, which recomputes density from its own surveyed zone area — the database owns ground truth — and stores, caches and fans it out.

Density bands (2.0 / 3.5 / 5.0 persons per m²) are published crowd-safety figures and live in code, deliberately not in editable config: an administrator under pressure at 2 a.m. must not be able to make a critical zone look safe by editing a row.

M3**Temple Command Center**

Solves: the operator's decision. One screen, ordered by what needs attention.

- **Live map** of zone polygons coloured by band, each carrying its reading age.
- **Alert feed** — one open alert per zone per rule, refreshed rather than duplicated while a condition persists. A zone sitting at 5.2 p/m² for ten minutes is one situation, not sixty rows; burying an alert is as dangerous as missing it.
- **Escalation** — an unacknowledged CRITICAL escalates visually after 60s, then pages the next role at 180s, and the delay is written to the audit log.
- **KPI strip** — throughput vs target, cameras online, open incidents, SLA state.
- **Replay scrubber** — reconstructs any past window from the time-series, for review.

M4**Emergency & Incident Management**

Solves: the response loop, with a clock on it.

- **SOS is never hard-blocked.** Rate limits exist everywhere else; a person pressing the button four times is frightened, not abusive, so an over-limit SOS attaches to their existing open incident instead of failing.
- **Offline SOS queues** on the phone and keeps the clock it started with, so an incident raised in a network dead zone is not back-dated to when the network returned.
- **Dispatch** ranks responder units by real distance (PostGIS), showing why each was suggested. An empty list explains itself rather than showing nothing.
- **SLA by severity** (critical = 3 min). Only incidents with *no responder assigned* can breach — once a unit is on the way the control room has done the thing being measured, and counting it as failure would push operators to dispatch anybody just to stop the clock.
- **Missing persons** — the highest-frequency real incident. Photos auto-purge 30 days after closure; every photo view is audit-logged.

M5 Queue Breach & Audit Ledger

Solves: accountability that survives pressure.

- A directional **tripwire** at a restricted gate records that an unauthorised crossing occurred, with a 10-second evidence clip.
- **Cross-referenced against pass scans** within ± 30 seconds — a pilgrim whose pass was scanned as they walked through is a pilgrim, and recording them would bury real events in noise.
- **Hash-chained.** Each record's hash covers the previous one, so altering or removing a row is detectable. Verified hourly; a detected break is written to the append-only audit log, so the discovery of tampering cannot itself be quietly removed.
- **Viewing a clip** requires re-authentication and a purpose note, and writes an access log row visible to the Administrator.
- **Redaction, not deletion.** Evidence can be removed with a mandatory written reason, but the record and its hash remain — a ledger whose rows can vanish is not a ledger.

THE ETHICS POSITION, STATED PLAINLY

The breach system's purpose is **accountability of a process, not surveillance of people**. It records that a rule was broken at a gate at a time. It does not identify who broke it — there is no column in the table that could — and leaves that question to lawful human process.

M6 Predictive Crowd Forecasting

Solves: acting before the density, not during it.

- LightGBM predicts density per zone at **30 / 60 / 90 minutes**, with a prediction interval.
- **A wide interval refuses to recommend action.** A prediction of 4.2 p/m² with a band from 1.1 to 7.0 is a model shrugging; acting on it is how operators learn to ignore the forecast strip entirely. It produces "go and look", never "hold a gate".
- **Only one horizon may raise an alert** (60 min by default). Alerting on all three would page an operator three times about one approaching surge, and the third page is the one that gets the feed muted.
- Recommendations carry concrete numbers — "*hold Gate 2 intake for 15 min*" — computed from the forecast by a rule, never composed by a model.
- Forecasts are **stored, not just computed**, with the predicted moment as a real column, so a scoring query can join a prediction to the reading that eventually arrived and say whether it was right.

M7 Pilgrim PWA

Solves: the pilgrim's own experience, on a cheap phone with no signal.

- **Marathi-first.** Marathi is the operational language; English is the translation, not the reverse.
- **Offline shell.** Pass, map, facility locations, ritual timings and emergency numbers all survive going offline — because the moment they matter most is the moment the network has failed.
- **Cacheable and non-cacheable are split deliberately.** Facility locations are safe to cache for a day. Live crowd data is served separately with its own timestamp and an explicit staleness flag, and the app must render it with its age. Never render stale crowd data as if it were live.
- **A no-JS pass card** renders the QR server-side, so a pass works on a phone that cannot run the app.

M8 Palkhi & Dindi Tracking

Solves: the other seventeen days. Every other module is about the last day of the Wari.

- **A Dindi is a group, not a person.** One designated volunteer phone reports for a few hundred walkers. A second phone reporting for the same Dindi is *refused* — a Palkhi in two places on a board that halt towns provision against is worse than no position at all.
- **Battery-aware reporting, decided by the server.** 60s at full charge, backing off to ten minutes at 25% and fifteen at 10%. The phone knows its battery; only the server knows the group is halted for the night with six days of walking left to power.
- **Pace is displacement over 90 minutes projected onto the route line** — not the GPS speed field, which reports the speed of the volunteer carrying the phone and spikes when he jogs to catch up.
- **An unusable pace produces no arrival estimate at all.** Not a default 3 km/h — a halt town cannot distinguish a guessed ETA from a measured one once it is a time on a screen, and will staff the kitchen for both the same way.
- **Early is treated as more dangerous than late.** Late means food going cold. Early means five hundred people walking into a town whose water tankers have not arrived — so a confident early arrival at a town not marked ready is the one CRITICAL rule in the table.
- **The readiness board shows two statuses and their disagreement.** What a coordinator declared, and what the provisioning actually supports against the head count walking towards them. A town marked "ready" with water for half the arrivals is exactly the failure this module exists to catch.

Solves: questions, in the pilgrim's own language — under a deliberately narrow contract.

- **Five read-only tools:** pass status, zone crowd, nearest facility, schedule, SOS draft. There is no cancel-pass tool, no set-threshold tool, no dispatch tool — so no prompt, injection or model error can reach one. **The tool list is the control;** a guardrail a model can talk its way past is not a guardrail.
- **The SOS tool drafts and does not send.** A language model must not be able to put a row in a control room's alarm queue — and equally must not be the thing that decides an alarm is unnecessary. It helps a frightened person find words; a human presses the button.
- **Emergencies are redirected before a model is involved.** "My father has collapsed" is caught by a keyword triage pass and answered with 108, the SOS button and one instruction. Not a refusal — a refusal is a dead end to a person in the worst minute of their life.
- **Every factual claim comes from a tool call in that turn.** When a tool returns nothing, the answer is "I don't know" plus the helpline. Never fabricate a wait time: a wrong number here sends a person into a crowd at the wrong moment.
- **It runs without a model.** With no API key or an unreachable provider, a keyword router answers from the same five tools with templated bilingual sentences, logged as `degraded`.

6 How it all works together

The modules above are not independent features. This is the loop they form during a real surge — the sequence that is the actual product.

- 1 **T+0:00 — Normal state.** All zones green. 38 of 40 cameras online. Throughput 5,800/hr against a 6,000 target. Pilgrims hold time-slotted passes with honest wait estimates.
- 2 **T+1:30 — A Palkhi arrival surge begins.** Zone C density climbs. Critically, the **stagnation index rises first** — the crowd is still dense but has stopped moving. The alert fires on stagnation before density reaches CRITICAL. *This is the technical beat of the whole project.*
- 3 **T+1:35 — The alert reaches the operator with an action.** Not "Zone C is at 4.1 p/m²", but rule **R-M2-03** : *"The queue has stalled while still dense. Find what is blocking the front before adding anyone behind it. Throttle intake to half until movement resumes."* In Marathi, with the rule id visible.
- 4 **T+2:15 — The forecast agrees and quantifies.** Zone C predicted at 4.6 p/m² in 30 minutes, interval 4.1–5.0 — narrow enough to act on. A recommendation card appears with a concrete diversion and a hold duration computed from the prediction.
- 5 **T+2:45 — The operator acts, and the system follows through.** Intake is held. Measured throughput drops below plan. The reslotting job detects the deviation and shifts downstream passes — and the pilgrim's phone buzzes with a revised slot. **One operator decision propagates to ten thousand phones without anyone re-typing anything.**
- 6 **T+3:30 — A restricted gate is breached.** A tripwire fires. The system checks for a pass scan within $\pm 30s$, finds none, and writes a breach record with a clip and a chain hash. A reviewer marks it. The ledger verifies.
- 7 **T+4:15 — A pilgrim raises an SOS in a dead zone.** The app queues it offline, keeping the timestamp it was pressed. On reconnect it dispatches, the nearest ambulance is suggested by distance, and the SLA clock runs from the original press.
- 8 **T+4:45 — The AI service is killed.** The map goes to UNKNOWN — not to green. Passes keep issuing, scanning keeps working, incidents and the ledger are untouched. The control room switches to radio reporting per the runbook. **The system degrades and says so.**

WHAT THE LAST STEP DEMONSTRATES

Anyone can demonstrate a happy path. The property worth showing is that when a component dies, the failure is *visible, bounded and survivable* — and that the screens tell the truth about what they no longer know.

7 Data model and information flow

Table group	Holds	Retention
zones, cameras, tripwires, gates, facilities	Spatial configuration; polygons, homographies	Permanent
density_readings (hypertable)	10-second aggregate per zone. No person data.	30d raw / 1y aggregate
slots, passes, pass_members	Capacity ledger and issued passes (phone as HMAC)	Season
incidents, incident_events, responders	Response timeline with SLA state	Season + review
breach_events, clip_access_log	Hash-chained ledger; every clip view	Clips 90d, records permanent
forecasts (hypertable)	Predictions with the predicted moment as a column	30d
dindis, dindi_pings, dindi_schedule, halt_towns	Group positions and halt plans	Season
assistant_turns	Every AI turn with its tool calls; numbers redacted	90d
audit_log	Every privileged action. Append-only by DB trigger.	7 years
contact_secrets	Encrypted raw phone numbers, purpose-scoped, TTL'd	30–210d by purpose

7.1 The phone-number rule, concretely

Section 12's data-minimisation requirement is implemented in three layers, not asserted in policy:

1. **HMAC on the entity.** `passes.holder_phone_hash` — keyed, so the hashes are not attackable by enumerating the ten-digit space.
2. **Raw only in** `contact_secrets`, Fernet-encrypted, with `purge_after` set at write time and a scheduled job enforcing it.
3. **Purpose is a column.** A number stored to notify a pass holder cannot be read by a code path looking for a Dindi leader.

8 Privacy, security and compliance

8.1 Architectural constraints — enforced as absences

Constraint	How it is guaranteed
No facial recognition, gait analysis or biometric templates	No model infers identity; no table could store one
No cross-camera re-identification	Track IDs are in-memory and discarded after aggregation
No video persisted	Frames processed in memory; sole exception is the 10s breach clip
No pilgrim location stored against identity	Position stays on the device; the map is served as polygons
No raw phone number on any entity row	HMAC only; raw is encrypted, purpose-scoped and TTL'd

8.2 Access control and application security

- Argon2id password hashing; JWT rotation with **refresh-token reuse detection**.
- MFA required for Administrator and System Admin, enforced at the dependency layer rather than per route.
- **Authorisation lives in exactly one file**. Routes ask for a *permission*, never a role, so `if role == "admin"` appears nowhere in the codebase. Adding a role is editing one dict.
- Append-only audit log protected by a database **trigger** — a grant can be changed by whoever holds the role; a trigger has to be dropped, and dropping it is a visible schema change.
- Strict CORS, CSP `default-src 'none'`, HSTS in production, parameterised queries throughout.
- `bandit` and `pip-audit` on every pull request; the app refuses to boot in production with development secrets still in place.

8.3 Notices generated

CCTV signage in Marathi, Hindi and English (with a separate, differently-worded board for restricted gates where a clip is recorded — posting the standard board there would be a false statement on a legal notice), plus consent copy for location, notifications and designated Dindi volunteers.

9 Degradation: what happens when things break

Section 11 requires a defined and tested manual mode for every module. This is the table that makes "graceful degradation" a specification rather than a hope.

Component fails	What still works	Manual mode
AI service	Passes, scanning, SOS, incidents, dispatch, breach ledger, pilgrim app	Map goes UNKNOWN (never green). Radio crowd reporting per zone every 10 min. Reslotting frozen.
Some cameras	Everything; affected zones drop confidence	One coverage alert per zone. Radio reporting for that zone only.
Redis	Everything — reads fall back to Postgres	None needed. Events lose 2s of freshness; jobs run without their lock and are idempotent.
Gemini / assistant	All other modules; assistant still answers	Deterministic templated answers over the same five tools, logged as degraded.
Dindi phone	Everything; that group shows <i>no</i> ETA	Call the leader (audited endpoint). Halt town plans from the printed schedule.
Network at a gate	Pass QR still validates — signature is checked locally	Scanner queues scans; reconciles on reconnect.
Core API / database	Pilgrim app offline shell; pass display; SMS SOS	Full paper mode: gate tallies on paper, admit on the printed slot time, SOS to the control-room SMS number.

THE RULE THAT GOVERNS EVERY ROW ABOVE

"Unknown" is a safe state; "clear" is not. Every degradation preserves that distinction. The console has **no manual density-entry field** by design — a typed-in number would be indistinguishable from a measured one on every downstream screen, which is precisely the confusion the whole system exists to prevent.

10 Operations and observability

- **Two families of metrics, deliberately separate.** Per-endpoint metrics answer *is the API healthy*. Per-zone-pipeline metrics answer *is the system still seeing the Wari*. Those can disagree — an API serving 200 OK on every route while forty cameras have gone dark is a green dashboard over a blind control room, and the gap between the two is where an outage hides.
- **Gauges go stale on purpose.** When a feed dies, its zone keeps its last density and the *reading-age* metric climbs without limit. Alerts fire on the age, never the density. A metric that zeroed itself when its feed died would render an unmeasured zone as an empty one.
- **Alerts fire on blindness, never on busyness.** Density appears in zero alert rules; reading-age appears in two. A dense zone is Tuesday of the Wari and already reaches an operator through the product's own

pipeline. Paging an engineer for it would train them to silence the pager that also carries "the cameras are dark".

- **Load testing asserts the thing that is invisible.** A stampede of 20,000 requests in 60 seconds must "queue, not collapse" — but a full slot answering *409 SLOT_FULL* to 19,000 phones is the system *working*. The real failure is confirming more seats than a slot holds, and every one of those requests returns 201. So a separate checker reads the ledger directly and verifies no oversubscription, no counter drift, and no erosion of the walk-in reserve.
- **Backup is snapshot plus PITR**, and the restore drill verifies that the restored *hash chain still verifies* — a backup path that silently corrupted evidence would defeat the ledger's purpose without anyone noticing.

11 Build status — honestly reported

A module is not complete because its files exist. This is what runs, what does not, and what is deferred.

Phase	Scope	State	Note
1	Foundation — schema, auth, RBAC, audit, CI	Done	Runs from one compose command
2	M1 Pass system — slots, QR, scan, reslotting	Done	With unit and API tests
3	M2 Crowd engine — sim, pipeline, zones, WS	Done	Sim engine allows validation without cameras
4	M3 Command centre — map, feed, KPIs, replay	Done	
5	M4 Incidents, SOS, dispatch, missing persons	Done	SLA sweep endpoint added late; verification pending
6	M5 Breach ledger, chain, review flow	Done	3 tamper-simulation tests need a fix (see below)
7	M7 Pilgrim PWA — offline, Marathi, a11y	Done	
8	M6 Forecasting — intervals, rules	Done	
9	M8 Palkhi tracking + AI assistant	Done	108 rule/guardrail tests, 22 API tests passing
10	Hardening — load test, runbook, DPIA, observability, backup	Done except drill	Restore drill written but not yet executed

11.1 Known open items

- **The restore drill has not been run.** Section 11 requires it to be "actually performed once". The scripts are written and reviewed against the real schema, but no drill has executed. The drill log is empty and says so. *It is not done until there is a row in it.*
- **Three breach tests fail** because the test tampers with the ledger via SQL to prove detection, but the anti-tamper database trigger blocks the write. The test and the guard contradict each other; the fix is in the test — disable the trigger around the tamper to honestly model an attacker with direct database access.

- **The SLA sweep endpoint is unverified.** Eleven tests referenced a route that did not exist; it has been implemented but not yet exercised against a live database.
- **Six DPIA findings need a named human**, two of them blocking: a Data Protection Officer with a published grievance channel, and written Marathi consent from designated Dindi volunteers.

12 What makes this different

Common approach	What WariVerse does instead
Alert on density thresholds	Alert on stagnation and counter-flow — the precursors that appear first
Show a number	Show a number with its age, source and confidence , and refuse to show one it cannot stand behind
AI recommends an action	A numbered rule recommends; the AI may only rephrase it
Assume connectivity	Offline-first pilgrim app, SMS fallback, locally-verifiable QR
Log for debugging	Hash-chained ledger designed to survive political pressure
Face recognition for security	Zero biometrics — deployable under the DPDP Act by construction
Cover the peak day	Cover the full eighteen days , including halt-town readiness
"It works" means the happy path	Every module has a defined, documented manual mode
Optimise for the operator	Optimise for the pilgrim without a smartphone too — a 25% walk-in reserve enforced in arithmetic

IN ONE PARAGRAPH

WariVerse turns the Wari from a crowd that is managed after it forms into a crowd that is anticipated. It spreads arrivals with honest, self-correcting time slots; it watches for the stalled-flow condition that precedes a crush rather than the density that follows it; it hands operators a numbered rule instead of a raw number; it records queue breaches in a ledger that resists exactly the pressure such records attract; and it does all of this without a single biometric, on a phone with no signal, in Marathi — and keeps running, visibly degraded and honestly labelled, when its own components fail.